

LAYER: 2_MesCore

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `**/obj/**`, `**/bin/**`, `**/*.png`, `**/*.jpg`, `**/node_modules/**`, `**/*.Designer.cs`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

```
Promatis.Net.MES.Configuration/Class1.cs
Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj
Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs
Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs
Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj
Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs
Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs
Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs
Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj
Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationHierarchyEngine.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/TechnologicalParameterCalcMethodBase.cs
```

```

Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitBase.cs
Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs
Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/ITechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/TechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/ITechnologicalParameterService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/TechnologicalParameterService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs
Promatis.Net.MES.Service/UnitService/UnitBaseService.cs
Promatis.Net.MES.Tests/AssemblyInfo.cs
Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj
Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs
Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs
Promatis.Net.MES.Tests/UnitBase/TestUnit.cs
Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs
Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs
Promatis.Net.MES.UI/_Imports.razor
Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj
Promatis.Net.MES.UI/UiModule.cs
Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

<file path="Promatis.Net.MES.Configuration/Class1.cs">

```
namespace Promatis.Net.MES.Configuration
```

```
{
    public class Class1
    {
    }
}
```

</file>

<file path="Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj">

```
<Project Sdk="Microsoft.NET.Sdk">
```

```

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>

```

```

    <ItemGroup>
        <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>

```

```

    </Project>
</file>

```

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs">

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;

```

```

using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Data;
{
public abstract class TechnologicalOperationParameterBaseConfiguration<T, TOperation, TParameter> :
IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationParameterBase<TOperation, TParameter> // <--- A05D 5CD0CT< C0;CäAC=8CR ?C
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C=A CD;D0 ?D 5CD>D$2D 0D"5C08D0 4D41C'8D >C$0C08D0 ?C @C <CTBD >C" =C >C
        builder.HasIndex(x => new { x.OperationId, x.ParameterId })
            .IsUnique();
    }

    // 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6ögB FVÆWFR•
    // A,,AC=;DäGC 5CÄ 7C ?C,,ADÄ 8Cr 2D'1Cä@C=8, CTAC'8 D44C ;CT=C AC <C AC$0CtL (ISoftDeletable.Deleted
    // A,, A, 5D ;C, 1D'; CÄ0C4:Cä CCD0C'5C0 AC < C0@C,,2D07C =C0KC' BCTEC0>C'>C48Dt5D :C,,9 C00D 0CÄ5D$@ (P
    builder.HasQueryFilter(x =>
        x.DeletedAt == null &&
        EF.Property<DateTime?>(x.Parameter, "DeletedAt") == null);
    }
}
}
</file>

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Data;
{
/// <summary>
/// A=C0DC,,3D4@C FC,,0 CD;D0 1C 7D² AC$0Ct8D
/// </summary>
/// <typeparam name="T"></typeparam>
public abstract class TechnologicalOperationUnitBaseConfiguration<T, TOperation> :
IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationUnitBase<TOperation> // <--- A05D 5CD0CT< D$8C0 >C05D 0Dd8C•
    where TOperation : DomainObject, ITechnologicalOperation
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C=A CD;D0 ?C @D² ?Cä;CT9 (A0@CT4CäBC$@C IC 5D" 4D41C'8D >C$0C08CR AC$0Ct5
        builder.HasIndex(x => new { x.OperationId, x.UnitId })
            .IsUnique();
    }

    // 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6ögB FVÆWFR•
    // BD8C'LD$@D45CÄ AC <D2 AC$0CtCDäID4N D CD"=CäAD$L, CTAC'8 Cä=C ?Cä<CTGCT=C :C : D44C ;CT=C00D0í
    // A,, A, 5D ;C, CCD0C'5C0> D 2D07C =C0>CR A C05C' >C >D CCD>C$0C08CR ...Væ-B'í
    // A,,AC0>C'LctCCT< EF.Property CD;D0 1CT7Cä?C AC0>C4> CD>D BD4?C : D$5C05C$KCÄ0C$=D'< D 2Cä9D BC$0C
    builder.HasQueryFilter(x =>
        x.DeletedAt == null &&
        EF.Property<DateTime?>(x.Unit, "DeletedAt") == null);
    }
}
}
</file>

<file path="Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.MES.Domain;
{
namespace Promatis.Net.MES.Data;
{
public class UnitBaseConfiguration : IEntityTypeConfiguration<UnitBase>
{
    public void Configure(EntityTypeBuilder<UnitBase> builder)
    {
        // A$:C'NDt0CT< D BD 0D$5C48Dä E B 4C'0 C$ACT9 C,,5D 0D EC,,8 UnitBase
        builder.UseTptMappingStrategy();
    }

    // A00D BD >C":C ?Cä;CT9D
}
}

```

```

        builder.Property(x => x.Kind).IsRequired();
        builder.Property(x => x.Type).IsRequired();
    }
}
</file>

<file path="Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

public class MesApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null) : ApplicationDbContext(options, configuration, serviceProvider)
{
    public DbSet<UnitOfMeasurement> UnitOfMeasurements => Set<UnitOfMeasurement>();
}
</file>

<file path="Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Data\Promatis.Net.Data.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Domain\Promatis.Net.MES.Domain.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

/// <summary>
/// A 1D BD 0C=BC0KC' 1C 7CäD'9 D$@C,,3C45D 8CT@C @DT8C, BCTEC0>C'>C48Dt5D :C,,E Cä?CT@C FC,,9.D
/// At0D'8D'0CTB D BD CC=BD4@D2 >D" =CT:Cä@D 5C=BC0>C4> CD>C 0C$;CT=C,,0 C0>CDCCT;Cä2 C, :Cä=D$@Cä;C,,@D45D" AD
/// </summary>
public abstract class TechnologicalOperationHierarchyTrigger<T, TOLink, TPLink> :
IBeforeSaveTrigger<T>
{
    where T : TechnologicalOperationBase<T, TOLink, TPLink>
    where TOLink : class
    where TPLink : class
    {
        public async Task HandleAsync(EntityCancelEventArgs<T> args)
        {
            T operation = args.Entity;

            // ATAC'8 Cä?CT@C FC,,0 C=D =CT2C 0 - C0@Cä?D4AC=0CT< C0@Cä2CT@C=C0
            if (!operation.ParentId.HasValue || operation.ParentId == Guid.Empty) return;

            // A,,7C$;CT:C 5CÄ 8Cr !B4 AB AD$0D$CD -4ÆV b 8 Name D >CD8D$5C'LD :Cä9 Cä?CT@C FC,,8D
            var parentInfo = await args.Context.Set<T>().Where(x => x.Id == operation.ParentId).Select(x => new { x.IsLeaf, x.Name }).FirstOrDefaultAsync();

```

```

Ð
if (parentInfo != null)Ð
{Ð
    // A,,ACô>C´LctCCT< CÔ0D, 4Cä<CT=CÔKC´ 4C$8Cd>C¢ 4C´O Cô@Cä2CT@C¤8 Cô@C 2C,,; C$;Cä6CT=CÔ>D BC,í
    // Aô5D 5CD0CT< D BC BD4A IsLeaf D >CD8D$5C´O, C$KD$0D"5CÔ=D´9 C,,7 C 0CtK.Ð
    if (parentInfo.IsLeaf)Ð
    {Ð
        args.Cancel = true;Ð
        args.ErrorMessage = $"AÔ0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3¢ "CTECÔ>C´>C48Dt5D :C O Cä?CT@C FC,,O '{
            $"CÔ5 CÄ>Cd5D" 1D´BDÄ 2C´>Cd5CÔ0 C" w. &VçD-æfòäæ ÖW0rÂ BC : C¤0C¢ >CÔ0
        }
        return;Ð
    }Ð
}Ð
}Ð
Ð
// AD>Cô>C´=C,,BCT;DÄ=C O Ct0D"8D$0 CÔ0 C,,7CÄ5CÔ5CÔ8CS¢ 5D ;C, >Cô5D 0Dd8Dò AD$0CÔ>C$8D$ADò C,,AD$>CÄÂ
// Cô@Cä2CT@Dô5CÄ 2 ChangeTracker C,,;C, ABÄ =CTB C´8 D2 =CT5 CD>Dt5D =C,,E DÔ;CT<CT=D$>C-
if (operation.IsLeaf)Ð
{Ð
    bool hasChildrenInDb = await args.Context.Set<T>()Ð
        .AnyAsync(x => x.ParentId == operation.Id);Ð
}Ð
Ð
if (!TechnologicalOperationHierarchyEngine.CanChangeLeafStatus(hasChildrenInDb,
operation.IsLeaf))Ð
{Ð
    args.Cancel = true;Ð
    args.ErrorMessage = $"AÔ0D CD,,5CÔ8CR FCT;CäAD$=CäAD$8 CD0CÔ=D´E: Aô5C$>Ct<Cä6CÔ> D 4CT;C BDÄ
        $"D$0C¢ :C : C$=D4BD 8 CÔ5CR CCD5 D >CD5D 6C BD O CD@D43C,,5 D$5DT=Cä;Cä3C
    }Ð
}Ð
}Ð
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.MES.Domain;Ð
Ð
namespace Promatis.Net.MES.Data;Ð
Ð
/// <summary>Ð
/// B$@C,,3C45D 4C´O Cô@Cä2CT@C¤8 D ?CTFC,,DC,,GCÔKDR ?D 0C$8C² 8CT@C @DT8C, Væ-D& 6Rí
/// </summary>Ð
public class UnitBaseHierarchyTrigger : IBeforeSaveTrigger<UnitBase>Ð
{Ð
    public async Task HandleAsync(EntityCancelEventArgs<UnitBase> args)Ð
    {Ð
        UnitBase unit = args.Entity;Ð
        Ð
        if (!unit.ParentId.HasValue || unit.ParentId == Guid.Empty) return;Ð
        Ð
        // Aô>C´CDt0CT< Kind C, æ ÖR @Cä4C,,BCT;Dò 4C´O C,,=DD>D <C BC,,2CÔ>C4> D >Cä1D"5CÔ8Dý
        var parentInfo = await args.Context.Set<UnitBase>()Ð
            .Where(x => x.Id == unit.ParentId)Ð
            .Select(x => new { x.Kind, x.Name })Ð
            .FirstOrDefaultAsync();Ð
        Ð
        if (parentInfo != null && !UnitHierarchyEngine.IsHierarchyValid(parentInfo.Kind, unit.Kind))Ð
        {Ð
            args.Cancel = true;Ð
            args.ErrorMessage = $"AÔ0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3¢ C JCT:D" w.Væ-Bäæ ÖW0r ‡.Væ-Bä¶-æGò' " ½
                $"CÔ5 CÄ>Cd5D" 1D´BDÄ 2C´>Cd5CÔ 2 '{parentInfo.Name}' ({parentInfo.Kind}).";Ð
        }Ð
    }Ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs">
using System.ComponentModel;Ð
Ð
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
/// <summary>Ð
/// AÄ5D$>CDK D 0D GCTBC BCTE. CÔ0D 0CÄ5D$@Cä2D
/// </summary>Ð
[Flags]Ð

```

```

public enum CalculationMethod
{
    [Description("A05 Ct0CD0C0"•Ý
    None = 0, // 00000000
}

[Description("AÄ0C0AC,,<C ;DÄ=Cä5 Ct=C GCT=C,,5 (MAX)")]
Max = 1,

[Description("AÄ8C08CÄ0C´LC0>CR 7C00Dt5C08CR „0"ä'"•Ý
Min = 2,

[Description("B @CT4C05CR 7C00Dt5C08CR „ dr'"•Ý
Avg = 4,

[Description("A05D 2Cä5 Ct=C GCT=C,,5 (FIRST)")]
First = 8,

[Description("A0>D ;CT4C05CR 7C00Dt5C08CR „Ä 5B'"•Ý
Last = 16,

[Description("A$ACR 7C00Dt5C08D0 „ ÄÄ'"•Ý
All = 32
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs">
using System.ComponentModel;
namespace Promatis.Net.MES.Domain.Interface;
public enum UnitKind
{
    [Description("B :C´0CDAC0D0 ;Cä3C,,AD$8C0")]
    Storage = UnitType.Storage | UnitType.Zone | UnitType.Rack | UnitType.Cell | UnitType.Crane,

    [Description("A0@Cä8Ct2Cä4D BC$5C0=C 0 Ct>C00")]
    Production = UnitType.Workshop | UnitType.Section | UnitType.Line | UnitType.Workstation |
    UnitType.MachineTool | UnitType.Table,

    [Description("B$@C =D ?Cä@D$=D´9 D47CT;")]
    Transport = UnitType.Vehicle | UnitType.Conveyor,

    [Description("A0>CD@C 7CD5C´5C08CR"•Ý
    Department = UnitType.Workshop | UnitType.Section | UnitType.Other,

    [Description("B 0C >Dt0D0 BCäGC0 / B0GCT9C0")]
    Position = UnitType.Cell | UnitType.Other
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs">
using System.ComponentModel;
namespace Promatis.Net.MES.Domain.Interface;
[Flags]
public enum UnitType
{
    None = 0,

    [Description("Bd5DR"•Ý
    Workshop = 1,

    [Description("B4GC AD$>C¢"•Ý
    Section = 2,

    [Description("A´8C08D0 0 Cä=C$5C"5D "•Ý
    Line = 4,

    [Description("B 0C >Dt5CR <CTAD$>")]
    Workstation = 8,

    [Description("B :C´0CB"•Ý
    Storage = 16,

    [Description("At>C00 DT@C =CT=C,,0")]

```

```

    Zone = 32,Ð
Ð
    [Description("B BCT;C´0Cb"•Ý
    Rack = 64,Ð
Ð
    [Description("B0GCT9C00 C 4D 5D 0 DT@C =CT=C,,0")]Ð
    Cell = 128,Ð
Ð
    [Description("A0@C = / A0>CDJCT<C08C0"•Ý
    Crane = 256,Ð
Ð
    [Description("B BC =Cä:")]Ð
    MachineTool = 512,Ð
Ð
    [Description("A$5D AD$0C0 0 !D$>C²"•Ý
    Table = 1024,Ð
Ð
    [Description("B$@C =D ?Cä@D$=Cä5 D @CT4D BC$>")]Ð
    Vehicle = 2048,Ð
Ð
    [Description("A 2D$>C0>CÄ=D´9 D$@C =D ?Cä@D$5D "•Ý
    Conveyor = 4096,Ð
Ð
    [Description("A0@CäGCT5")]Ð
    Other = 8192Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs">
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public interface ITechnologicalOperationÐ
{Ð
    Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs">
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public interface ITechnologicalParameterÐ
{Ð
    Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
Ð
    <ItemGroup>Ð
        <ProjectReference Include="..\..\Core\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />Ð
    </ItemGroup>Ð
Ð
</Project>
</file>

<file path="Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
Ð
    <ItemGroup>Ð
        <ProjectReference Include="..\..\Core\Domain\Promatis.Net.Domain.csproj" />Ð
        <ProjectReference Include="..\
\Promatis.Net.MES.Domain.Interface\Promatis.Net.MES.Domain.Interface.csproj" />Ð
    </ItemGroup>Ð

```

```
Ð
</Project>
</file>
```

```
<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$0Dò BCTECô>C´>C48Dt5D :C 0 Cä?CT@C FC,,0.Ð
/// A00D ;CT4D45D" GC,,AD$>CR =CR04Cd5C05D 8C¢ 4CT@CT2Câ !B4 ABÂ CC 8D 0Dò :Cä=DD;C,, :D$K CÄ0C0?C,,=C40,Ð
/// C, ?D 5CD>D BC 2C´OCTB D BD >C4> D$8C08Ct8D >C$0C0=D´9 C,,=D$5D DCT9D 4C´O C 8Ct=CTA-C´>C48C¤8.Ð
/// </summary>Ð
public abstract class TechnologicalOperationBase<T, TOLink, TPLink> : ReferenceTreeBase<T>, // <- A¤ A,, "A,, 'AT
    ITechnologicalOperationÐ
    where T : TechnologicalOperationBase<T, TOLink, TPLink>Ð
    where TOLink : classÐ
    where TPLink : classÐ

{Ð
    public bool IsLeaf { get; set; } = true;Ð
Ð
    public virtual ICollection<TOLink> UnitLinks { get; set; } = new List<TOLink>();Ð
    public virtual ICollection<TPLink> ParameterLinks { get; set; } = new List<TPLink>();Ð
}
</file>
```

```
<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs">
using FluentValidation;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Promatis.Net.MES.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
// A$0C´8CD0D$>D >C05D 0Dd8C, ,,>D BC 5D$ADò 1CT7 C,,7CÄ5C05C08C´•
public abstract class TechnologicalOperationBaseValidator<T> : ReferenceTreeBaseValidator<T>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, ITechnologicalOperation // <- A¤ A,, "A,, 'AT!A¤ AR B B A$ A
{Ð
    protected TechnologicalOperationBaseValidator() : base()Ð
    {Ð
        // A 0Ct>C$KC´ &Vfw&Væ6T& 6Uf Æ-F F÷" CCd5 Cò@Cä2CT@C,,; Cä1D"8CR ?Cä;Dò ,, -BÂ æ ÖR•
        // At4CTADÂ <D² ?C,,HCT< Cò@C 2C,,;C AD$@Cä3Cä 4C´O D$5DT=Cä;Cä3C,,GCTAC¤8DR >C05D 0Dd8C•
Ð
        RuleFor(x => x.Code)Ð
            .NotEmpty()Ð
            .WithMessage("A¤>CB >C05D 0Dd8C, >C 0Ct0D$5C´5C0â"•
            .MaximumLength(50)Ð
            .WithMessage("A¤>CB >C05D 0Dd8C, =CR 4Cä;Cd5C0 ?D 5C$KD,,0D$L 50 D 8CÄ2Cä;Cä2.");Ð
    }Ð
}
</file>
```

```
<file path="Promatis.Net.MES.Domain/TechnologicalOperation/
TechnologicalOperationHierarchyEngine.cs">
using System.Reflection;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
public static class TechnologicalOperationHierarchyEngineÐ
{Ð
    /// <summary>Ð
    /// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,, : Cò@C 2CDK: Cò@Cä2CT@D05D"Â 2C ;C,,4C0> C´8 C$;Cä6CT=C,,5 CD>Dt5D =CT9 Cä?C
    /// A00CÄ5D BC$> C ;Cä:C,,@D45D" ACä7CD0C08CR 2CTBCä: C$=D4BD 8 D$5D <C,,=C ;DÄ=D´E Cä?CT@C FC,,9 (IsLeaf =
    /// B$8C08Ct0Dd8Dò ?CT@CT2CT4CT=C =C GC,,AD$KC´ :Cä=D$@C :D" 4CT@CT2C BÄ GD$> Cò>C´=CäAD$LDâ CD BD 0C00
    /// </summary>Ð
    /// <typeparam name="T">A¤>C0:D 5D$=D´9 D$8Cò BCTECô>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8</typeparam>Ð
    /// <param name="parent">B >CD8D$5C´LD :C 0 Cä?CT@C FC,,0</param>Ð
    /// <param name="childIsLeaf">Aô@C,,7C00C¢ BCä3CäÂ OC$;D05D$ADò ;C, ACä7CD0C$0CT<D´9/C05D 5CÄ5D"0CT<D´9 D
    public static bool IsHierarchyValid<T>(T? parent, bool childIsLeaf)Ð
        where T : class, ITreeNode<T>Ð
    {Ð
        // 1. ATAC´8 D >CD8D$5C´O C05D"Â <D² ACä7CD0CT< C¤>D =CT2Cä9 D0;CT<CT=D" r MD$> C$ACT3CD0 C$0C´8CD=Cä
        if (parent == null)Ð
    }Ð
}
```

```

        {
            return true;
        }
    }

    // 2. Bt5D 5Cr @CTDC`5C=AC,,N (C,,;C, :C AD" : C,,=D$5D DCT9D C, CTAC`8 C K Cä= C KC²' 8Ct2C`5C=θCT< Cθ@
    // B$0C¢ :C : T C00D ;CT4D45D$ADθ >D" FV6†æöÆöv-6 Ä÷ W& F-öä& 6RÂ AC$>C"AD$2Cä -4ÆV b BC < C40D θC0BC
    PropertyInfo? isLeafProperty = typeof(T).GetProperty("IsLeaf");
    if (isLeafProperty != null)
    {
        bool parentIsLeaf = (bool)isLeafProperty.GetValue(parent)!;
        if (parentIsLeaf)
        {
            return false; // B$5D <C,,=C ;DÄ=D`9 D47CT; C05 CÄ>Cd5D" 1D`BDÄ @Cä4C,,BCT;CT<D
        }
    }

    return true;
}

/// <summary>
/// Að@Cä2CT@D05D"Â @C 7D 5D,,5C0> C`8 C05D 5C=;DäGC BDÄ DC`0C2 -4ÆV b C D CD"5D BC$CDäICT9 Cä?CT@C FC,,8.D
/// ATAC`8 D2 >C05D 0Dd8C, CCd5 DD8Ct8Dt5D :C, 5D BDÄ 4CäGCT@C08CR 2CTBC=8 C" At#, C0@CT2D 0D"0D$L CTQ C
/// </summary>
public static bool CanChangeLeafStatus(bool hasChildren, bool requestedIsLeaf)
{
    if (hasChildren && requestedIsLeaf)
    {
        return false;
    }

    return true;
}
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
D
namespace Promatis.Net.MES.Domain;
D
/// <summary>
/// B 2D07D4ND`0D0 AD4IC0>D BDÄ ` C0>C48CR0:Cä0<C0>C48CÉ² <CT6CDC D$5DT=Cä;Cä3C,,GCTAC=8CÄ8 Cä?CT@C FC,,OCÄ8 C,
/// Aä?D 5CD5C`OCTB D$@CT1Cä2C =C,,0 C¢ 7C ?Cä;C05C08Dä ?C @C <CTBD >C" 4C`O C= >C0:D 5D$=Cä9 Cä?CT@C FC,,8.D
/// </summary>
public abstract class TechnologicalOperationParameterBase<TOperation, TParameter> : DomainObject,
ISoftDeletable
{
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
    {
        public Guid OperationId { get; set; }

        /// <summary>
        /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>C' >C05D 0Dd8C,1
        /// </summary>
        public virtual TOperation Operation { get; set; } = null!;

        public Guid ParameterId { get; set; }

        /// <summary>
        /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>CÄC C00D 0CÄ5D$@D21
        /// </summary>
        public virtual TParameter Parameter { get; set; } = null!;

        /// <summary>
        /// Að@C,,7C00C¢ BCä3CÄâ GD$> C00D 0CÄ5D$@ Cä1D07C BCT;CT= CD;D0 7C ?Cä;C05C08D01
        /// </summary>
        public bool IsRequired { get; set; } = false;

        /// <summary>
        /// A0>CÄ8C00C`LC0>CR 7C00Dt5C08CR ?C @C <CTBD 0 C0> D4<Cä;Dt0C08Dä1
        /// </summary>
        public string? DefaultValue { get; set; }
    }
}

```

```

    /// <summary>ð
    /// AÄ8C08CÄ0C´LCÔ> CD>C0CD BC,,<Cä5 Ct=C GCT=C,,5.ð
    /// </summary>ð
    public double? MinValue { get; set; }ð
ð
    /// <summary>ð
    /// AÄ0C0AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7C00Dt5C08CRı
    /// </summary>ð
    public double? MaxValue { get; set; }ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalOperationParameterBaseValidator<T, TOperation, TParameter> :
DomainObjectValidator<T>ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>ð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : TechnologicalParameterBaseð
{ð
    protected TechnologicalOperationParameterBaseValidator()ð
    {ð
        RuleFor(x => x.OperationId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8C00D$>D >C05D 0Dd8C, >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.ParameterId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8C00D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dô7C BCT;CT=.");ð
ð
        // A0@CÄAD Ô?Cä;CT2C 0 C$0C´8CD0Dd8Dó C :D 8CÄ0C´LCÔ>CR 7C00Dt5C08CR =CR <Cä6CTB C KD$L CÄ5C0LD,,5 C
        RuleFor(x => x.MaxValue)ð
            .GreaterThanOrEqualTo(x => x.MinValue)ð
            .When(x => x.MinValue.HasValue && x.MaxValue.HasValue)ð
            .WithMessage("AÄ0C0AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7C00Dt5C08CR =CR <Cä6CTB C KD$L CÄ5C0LD,,5 CÄ8C08C
    }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
    /// <summary>ð
    /// A 1D BD 0C0BC00Dò 1C 7C 4C´O D 2Dô7C, ,,@C AD,,8D OCT<C 0>ð
    /// </summary>ð
    public abstract class TechnologicalOperationUnitBase<TOperation> : DomainObject, ISoftDeletableð
        where TOperation : DomainObject, ITechnologicalOperationð
    {ð
        public Guid OperationId { get; set; }ð
    ð
        /// <summary>ð
        /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : Cä?CT@C FC,,8 (C CCD5D" 7C :D KD$> C0>CÔ:D 5D$=D´< D$8C0>CÄ 2 MDM)
        /// </summary>ð
        public virtual TOperation Operation { get; set; } = null!;ð
    ð
        public Guid UnitId { get; set; }ð
    ð
        /// <summary>ð
        /// A0@Dô<C 0 D AD´;C00 C00 C 0Ct>C$KC´ :C´0D A Cä1Cä@D44Cä2C =C,,0. ð
        /// EF Core C 2D$>CÄ0D$8Dt5D :C, AC$0Cd5D" MD$> D BC 1C´8Dd5C´ >C >D CCD>C$0C08Dò ?Cä ACä3C´0D,,5C08Dô<.ð
        /// </summary>ð
        public virtual UnitBase Unit { get; set; } = null!;ð
    ð

```

```

    public int Priority { get; set; } = 1;
}
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/
TechnologicalOperationUnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

/// <summary>
/// A 0Ct>C$KC' 2C ;C,,4C BCä@ D 2Dô7C•
/// </summary>
public abstract class TechnologicalOperationUnitBaseValidator<T, TOperation> :
DomainObjectValidator<T>
    where T : TechnologicalOperationUnitBase<TOperation>
    where TOperation : DomainObject, ITechnologicalOperation
{
    protected TechnologicalOperationUnitBaseValidator()
    {
        RuleFor(x => x.OperationId)
            .NotEmpty()
            .WithMessage("A,,4CT=D$8DD8C=0D$>D >Cô5D 0Dd8C, >C 0Ct0D$5C'5C0â""½

        RuleFor(x => x.UnitId)
            .NotEmpty()
            .WithMessage("A,,4CT=D$8DD8C=0D$>D >C >D CCD>C$0C08Dò >C 0Ct0D$5C'5C0â""½

        RuleFor(x => x.Priority)
            .InclusiveBetween(1, 10)
            .WithMessage("Aô@C,,>D 8D$5D" >C >D CCD>C$0C08Dò 4Cä;Cd5C0 1D'BDÂ 2 CD8C ?C 7Cä=CR >D" 4Câ â""
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

/// <summary>
/// A 0Ct>C$KC' BCTECô>C'>C48Dt5D :C,,9 C00D 0CÄ5D$@ (D ?D 0C$>Dt=C,,: DT0D 0C=BCt@C,,AD$8C¢ ?D >Dd5D ACä2).D
/// </summary>
public abstract class TechnologicalParameterBase : ReferenceBase, ITechnologicalParameter
{
    /// <summary>
    /// A$=CTHC08C' :C'Ndr =C ACô@C 2CäGC08C¢ 5CD8C08Db 8Ct<CT@CT=C,,0.D
    /// </summary>
    public Guid? UnitOfMeasurementId { get; set; }

    public virtual UnitOfMeasurement? UnitOfMeasurement { get; set; }

    /// <summary>
    /// B$8Cò 4C =C0KDR ?C @C <CTBD 0 (C00C0@C,,<CT@: Bt8D ;CâÂ !D$@Cä:C Â D4;CT2Câ'1
    /// </summary>
    public string DataType { get; set; } = "Numeric";

    /// <summary>
    /// B 0Ct@CTHCT=C0KCR <CTBCä4D² @C AD GE BC
    /// </summary>
    public CalculationMethod AllowedMethods { get; set; } = CalculationMethod.None;
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;

namespace Promatis.Net.MES.Domain;

```

```

public abstract class TechnologicalParameterBaseValidator<T> : ReferenceBaseValidator<T>ð
    where T : TechnologicalParameterBaseð
{ð
    private static readonly string[] AllowedDataTypes = ["Numeric", "String", "Boolean",
"DateTime"];ð
ð
    protected TechnologicalParameterBaseValidator() : base()ð
    {ð
        // 1. A$0C'8CD0Dd8D0 5CD8C08DdK C,,7CÄ5D 5C08D0 „4CäAD$CC0=C Â BC : C=0C¢ B r MD$> TechnologicalParame
RuleFor(x => x.UnitOfMeasurement)ð
        .NotNull()ð
        .WithMessage("AT4C,,=C,,FC 8Ct<CT@CT=C,,O C05 CÄ>Cd5D" 1D'BDÂ çVÆÂâ""½
ð
        // 2. A$0C'8CD0Dd8D0 BC,,?C 4C =C0KD]
RuleFor(x => x.DataType)ð
        .NotEmpty()ð
        .WithMessage("B$8C0 4C =C0KDR ?C @C <CTBD 0 Cä1D07C BCT;CT=.")ð
        .Must(type => AllowedDataTypes.Contains(type))ð
        .WithMessage($"A05CD>C0CD BC,,<D'9 D$8C0 4C =C0KDRâ C 7D 5D,,5C0=D'5 Ct=C GCT=C,,O: {string.Join(",
    }ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter> :
DomainObject, ISoftDeletableð
    where TUnit : UnitBaseð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : TechnologicalParameterBaseð
{ð
    public Guid UnitId { get; set; }ð
ð
    /// <summary>ð
    /// Bd5DT>C$0D0 5CD8C08Dd0ð
    /// </summary>ð
    public virtual TUnit Unit { get; set; } = null!;ð
ð
    public Guid TechnologicalOperationId { get; set; }ð
ð
    /// <summary>ð
    /// B$5DT=Cä;Cä3C,,GCTAC=0D0 >C05D 0Dd8Dý
    /// </summary>ð
    public virtual TOperation TechnologicalOperation { get; set; } = null!;ð
ð
    public Guid TechnologicalParameterId { get; set; }ð
ð
    /// <summary>ð
    /// B$5DT=Cä;Cä3C,,GCTAC=8C' ?C @C <CTBD
    /// </summary>ð
    public virtual TParameter TechnologicalParameter { get; set; } = null!;ð
ð
    /// <summary>ð
    /// AÄ5D$>CB @C ADt5D$0ð
    /// </summary>ð
    public CalculationMethod CalculationMethod { get; set; } = CalculationMethod.None;ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð

```

```

public abstract class TechnologicalParameterCalcMethodBaseValidator<T, TUnit, TOperation,
TParameter> : DomainObjectValidator<T>ð
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>ð
    where TUnit : UnitBaseð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : TechnologicalParameterBaseð
{ð
    protected TechnologicalParameterCalcMethodBaseValidator()ð
    {ð
        RuleFor(x => x.UnitId)ð
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C¤0D$>D  FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.TechnologicalOperationId)ð
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C¤0D$>D  BCTECÔ>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8 Cä1D07C BCT;CT=
ð
        RuleFor(x => x.TechnologicalParameterId)ð
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C¤0D$>D  BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1D07C BCT;
ð
        // A0@Cä2CT@D05CÄÂ GD$> CÄ5D$>CB @C Adt5D$0 C KC² OC$=Cä 2D´1D 0C0 ?Cä;DÄ7Cä2C BCT;CT<ð
        RuleFor(x => x.CalculationMethod)ð
            .NotEqual(CalculationMethod.None)ð
            .WithMessage("A05Cä1DT>CD8CÄ> C$KC @C BDÄ :Cä@D 5C¤BC0KC´ <CTBCä4 D 0D GCTBC ?C @C <CTBD >C"a""½
    }ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs">
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalParameterValueBase<TUnit, TParameter> : DomainObjectð
    where TUnit : UnitBaseð
    where TParameter : TechnologicalParameterBaseð
{ð
    public Guid UnitId { get; set; }ð
ð
    /// <summary>ð
    /// Bd5DT>C$0D0 5CD8C08Dd0 (C,,AD$>Dt=C,,: CD0C0=D´E)ð
    /// </summary>ð
    public virtual TUnit Unit { get; set; } = null!;ð
ð
    public Guid TechnologicalParameterId { get; set; }ð
ð
    /// <summary>ð
    /// B$5DT=Cä;Cä3C,,GCTAC¤8C´ ?C @C <CTBD
    /// </summary>ð
    public virtual TParameter TechnologicalParameter { get; set; } = null!;ð
ð
    /// <summary>ð
    /// B KD >CR ,3D 0Ct=Cä5") Ct=C GCT=C,,5, C05D 5CD0C0=Cä5 CäB SCADA/IoT C,,;C, >C05D 0D$>D 0ð
    /// </summary>ð
    public string Value { get; set; } = string.Empty;ð
ð
    /// <summary>ð
    /// B$>Dt=Cä5 C$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 (Time-Series CÄ5D$:C •
    /// </summary>ð
    public DateTime Date { get; set; } = DateTime.UtcNow;ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/
TechnologicalParameterValueBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalParameterValueBaseValidator<T, TUnit, TParameter> :
DomainObjectValidator<T>ð
    where T : TechnologicalParameterValueBase<TUnit, TParameter>ð
    where TUnit : UnitBaseð
    where TParameter : TechnologicalParameterBaseð
{ð
    protected TechnologicalParameterValueBaseValidator()ð
    {ð

```

```

        RuleFor(x => x.UnitId)()
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D  FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½
D
        RuleFor(x => x.TechnologicalParameterId)()
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D  BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1D07C BCT;
D
        RuleFor(x => x.Value)()
            .NotNull().WithMessage("At=C GCT=C,,5 C00D 0CÄ5D$@C =CR <Cä6CTB C KD$L null.")()
            // B 0Ct@CTHC 5CÄ ?D4AD$CDä AD$@Cä:D2Â BC : C=0C¢ 4C´0 String-C00D 0CÄ5D$@Cä2 D0BCä <Cä6CTB C KD$
            // C0> D41C,,@C 5CÄ AC´CDt0C"=D´5 C0@Cä1CT;D² ?Cä :D 0D0<D
            .Must(val => val == null || val.Trim() == val)()
            .WithMessage("At=C GCT=C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C´0CÄ8."
D
        RuleFor(x => x.Date)()
            .Must(date => date <= DateTime.UtcNow.AddSeconds(5))()
            .WithMessage("A$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 C05 CÄ>Cd5D" 1D´BDÄ 2 C CCDCD"5CÄâ""½
    }()
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBase.cs">
using Promatis.Net.Domain;()
using Promatis.Net.Domain.Interface;()
using Promatis.Net.MES.Domain.Interface;()
()
namespace Promatis.Net.MES.Domain;()
()
/// <summary>()
/// A 1D BD 0C=BC00D0 1C 7C 4C´0 C$ACTE Cä@C40C08Ct0Dd8Cä=C0KDR 8 C0@Cä8Ct2Cä4D BC$5C0=D´E CT4C,,=C,,F.D
/// </summary>()
public abstract class UnitBase : ReferenceTreeBase<UnitBase>, IAudit()
{()
    public UnitKind Kind { get; init; }()
    public required UnitType Type { get; set; }()
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs">
using FluentValidation;()
using Promatis.Net.Domain;()
()
namespace Promatis.Net.MES.Domain;()
()
/// <summary>()
/// A 1D BD 0C=BC0KC´ 2C ;C,,4C BCä@ CD;D0 1C 7Cä2Cä3Cä :C´0D AC Væ-D& 6R1
/// </summary>()
/// <typeparam name="T">B$8C0Ä =C AC´5CDCCT<D´9 CäB UnitBase.</typeparam>()
public abstract class UnitBaseValidator<T> : ReferenceBaseValidator<T>()
    where T : UnitBase()
{()
    protected UnitBaseValidator() : base()()
    {()
        // A0@Cä2CT@C=0, DtBCä 2 Type C05 C0@C,,HC´> C0CD BCä5/CD5DD>C´BC0>CR 7C00Dt5C08CR DC´0C4>C-
        RuleFor(x => x.Type).NotEmpty();()
D
        // A0@Cä2CT@C=0 D >CäBC$5D$AD$2C,,0 A=0D$5C4>D 8C, ,,¶-æB´ 8 B$8C00 (Type) C00 CäAC0>C$5 C 8D$>C$KDR <C
        RuleFor(x => x)()
            .Must(u => ((int)u.Kind & (int)u.Type) != 0)()
            .WithMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }()
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs">
using Promatis.Net.MES.Domain.Interface;()
()
namespace Promatis.Net.MES.Domain;()
()
public static class UnitHierarchyEngine()
{()
    /// <summary>()
    /// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Ä 2C ;C,,4C0> C´8 C$;Cä6CT=C,,5 D 5C 5C0:C 2 D >
    /// A,,AC0>C´LctCCTBD 0 C00CÄ5D BC$> C=0C¢ 2 B #A -D$@C,,3C45D 0DRÄ BC : C, 2 UI-C$0C´8CD0Dd8C,i
    /// </summary>()
    public static bool IsHierarchyValid(UnitKind parentKind, UnitKind childKind)()
    {()

```

```

        // 1. B$5D <C,,=C ;DÄ=D´9 D47CT; Position CÔ8C¤>C44C =CR <Cä6CTB C KD$L D >CD8D$5C´5Cí
        if (parentKind == UnitKind.Position) return false;ð
ð
        // 2. AD5CÔ0D BC <CT=D" <Cä6CTB D >CD5D 6C BDÂ 2D Q, C¤@Cä<CR ÷6–F–öí
        if (parentKind == UnitKind.Department) return childKind != UnitKind.Position;ð
ð
        // 3. B ?CTFC,,0C´8Ct8D >C$0CÔ=D´5 Ct>CÔK C,,7Cä;C,,@Cä2C =D² 4D CC2 >D" 4D CC40, CÔ> CÄ>C4CD" ACä4CT@Cd
        if (parentKind is UnitKind.Production or UnitKind.Transport or UnitKind.Storage)ð
        {ð
            return childKind == parentKind || childKind == UnitKind.Position;ð
        }ð
ð
        return true;ð
    }ð
ð
    /// <summary>ð
    /// A$>Ct2D 0D"OCTB D ?C,,ACä: Aä Aä B AÔ B´% D$8CÔ>C" ...Væ–EG– R´ 4C´O C$KCÔ0CD0DäICT3Cä ACô8D :C 2 UI,ð
    /// CäACÔ>C$KC$0DôADÄ =C C BCT3Cä@C,,8 (UnitKind) D >CD8D$5C´LD :Cä3Cä Cct;C Ä 2 C¤>D$>D KC´ <D² ACT9Dt0
    /// </summary>ð
    public static IEnumerable<UnitType> GetAllowedChildTypes(UnitKind parentKind)ð
    {ð
        var allTypes = (UnitType[])Enum.GetValues(typeof(UnitType));ð
        var allowedTypes = new List<UnitType>();ð
ð
        foreach (UnitType type in allTypes)ð
        {ð
            if (type == UnitType.None) continue;ð
ð
            // 1. B =C GC ;C =C ECä4C,,<, C¤ :C :Cä<D2 ?CäBCT=Dd8C ;DÄ=Cä<D2 ¶–æB >D$=CäAC,,BD O DÔBCäB D$8Còí
            // B :C =C,,@D45CÄ 2D 5 C¤0D$5C4>D 8C,Ä GD$>C K CÔ>CÔ0D$L, C" :C :D4N CÄ0D :D2 ?Cä?C 4C 5D" MD$>D"
            foreach (UnitKind potentialChildKind in (UnitKind[])Enum.GetValues(typeof(UnitKind)))ð
            {ð
                // ATAC´8 D$8Cò ?Cä1C,,BCä2Cä 2DT>CD8D" 2 DÔBD2 :C BCT3Cä@C,,ND
                if (((int)potentialChildKind & (int)type) != 0)ð
                {ð
                    // A, 5D ;C, ?D 0C$8C´0 C,,5D 0D EC,,8 B At AT(A .B" 2C´>Cd8D$L DÔBD2 :C BCT3Cä@C,,N C" BCT
                    if (IsHierarchyValid(parentKind, potentialChildKind))ð
                    {ð
                        if (!allowedTypes.Contains(type))ð
                        {ð
                            allowedTypes.Add(type);ð
                        }ð
                    }ð
                }ð
            }ð
        }ð
    }ð
ð
    return allowedTypes;ð
}
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs">
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// AD>CÄ5CÔ=C O D CD"=CäAD$L CT4C,,=C,,FD² 8Ct<CT@CT=C,,O (AÔ!A,´ 4C´O D$5DT=Cä;Cä3C,,GCTAC¤8DR ?C @C <CTBD >C"
/// </summary>ð
public class UnitOfMeasurement : ReferenceBaseð
{ð
    // B 2Cä9D BC$0 Id, Code, Name, Description CÔ0D$8C$=Cä CCÔ0D ;CT4Cä2C =D² >D" &VfW&Væ6T& 6Rí
    // Code – C¤@C BC¤>CR >C >Ct=C GCT=C,,5 (CÔ0CÔ@C,,<CT@, "°C", "CÄ<", "Cä1/CÄ8CÔ"´í
    // Name – CÔ>C´=Cä5 CÔ0C,,<CT=Cä2C =C,,5 (CÔ0CÔ@C,,<CT@, "A4@C 4D4A Bd5C´LD 8Dò"Ä $ C,,;C´8CÄ5D$@").ð
}
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public class UnitOfMeasurementValidator : ReferenceBaseValidator<UnitOfMeasurement>ð
{ð

```

```

// A05D 5Că?D 5CD5C'0CT< C 0Ct>C$CDâ @CT3D4;Dô@CαC C00 D 0D HC,,@CT=C0CDâ ,,@D4A/C'0D"Â FC,,DD K, C0@Că1CT;D
protected override string CodeRegexPattern => @"^[\p{L}\p{N}_\.\^\/*\-\s\%]*$";D
D
protected override string CodeValidationMessage => "Aα@C BCα>CR >C >Ct=C GCT=C,,5 CĂ>Cd5D" ACă4CT@Cd0D$L C
D
public UnitOfMeasurementValidator() : base()D
{D
    // A0@C 2C,,;Că æ÷DV× G' 8 Matches D46CR >D$@C 1CăBC ND" 2 C 0Ct>C$>CÂ :C'0D ACR ?Că =C HCT<D2 =Că2Că<
D
    RuleFor(x => x.Code)D
        .NotEmpty().WithMessage("Aα@C BCα>CR >C >Ct=C GCT=C,,5 Că1D07C BCT;DĂ=Că 4C'0 Ct0C0>C'=CT=C,,0.")D.
        .MaxLength(15).WithMessage("Aα@C BCα>CR >C >Ct=C GCT=C,,5 C05 CD>C'6C0> C0@CT2D'HC BDĂ R AC,,<
D
    RuleFor(x => x.Name)D
        .MaxLength(100).WithMessage("Aô>C'=Că5 C00C,,<CT=Că2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 C05 CD>C'
    }D
}
</file>

<file path="Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
    <PropertyGroup>D
        <TargetFramework>net10.0</TargetFramework>D
        <ImplicitUsings>enable</ImplicitUsings>D
        <Nullable>enable</Nullable>D
    </PropertyGroup>D
D
    <ItemGroup>D
        <ProjectReference Include="..\..\Core\Service\Promatis.Net.Service.csproj" />D
        <ProjectReference Include="..\Promatis.Net.MES.Data\Promatis.Net.MES.Data.csproj" />D
    </ItemGroup>D
D
</Project>
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs">
using Promatis.Net.Domain;D
using Promatis.Net.MES.Domain;D
using Promatis.Net.MES.Domain.Interface;D
using Promatis.Net.Service;D
D
namespace Promatis.Net.MES.Service;D
D
public interface ITechnologicalOperationParameterService<T, TOperation, TParameter> :
IBaseService<T, Guid>D
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>D
    where TOperation : DomainObject, ITechnologicalOperationD
    where TParameter : TechnologicalParameterBaseD
{D
    /// <summary>D
    /// Aô>C'CDt8D$L C$ACR ?C @C <CTBD K, C0@C,,2D07C =C0KCR : Cα>C0:D 5D$=Că9 Că?CT@C FC,,8D
    /// </summary>D
    Task<List<T>> GetByOperationIdAsync(Guid operationId);D
D
    /// <summary>D
    /// Aô>C'CDt8D$L D$>C'LCα> Că1D07C BCT;DĂ=D'5 C00D 0CĂ5D$@D² 4C'0 Că?CT@C FC,,8D
    /// </summary>D
    Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId);D
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Domain;D
using Promatis.Net.MES.Domain;D
using Promatis.Net.MES.Domain.Interface;D
using Promatis.Net.Service;D
D
namespace Promatis.Net.MES.Service;D
D
public abstract class TechnologicalOperationParameterService<T, TOperation, TParameter, TContext>(D
    IDbContextFactory<TContext> contextFactory)D
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalOperationParameterService<T,

```

```

TOperation, TParameter>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByOperationIdAsync(Guid operationId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.OperationId == operationId)
            .Include(x => x.Parameter) // B @C 7D2 ?Cä4D$OC48C$OCT< D ?D 0C$>Dt=C,, Cö0D 0CÄ5D$@Cä2 CD;Dö T•
            .AsNoTracking()
            .ToListAsync();
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
ITechnologicalOperationService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalOperationService<T, TLink> : ITreeService<T>
    where T : ReferenceTreeBase<T>, ITechnologicalOperation,
Promatis.Net.Domain.Interface.IDomainObjectHasKey<Guid>
    where TLink : class
{
    /// <summary>
    /// Aö>C`CDt8D$L D ?C,,ACä: D 0Ct@CTHCT=Cö>C4> CälCä@D44Cä2C =C,,O (Dä=C,,BCä2) CD;Dö :Cä=C@CTBCö>C' BCTECö
    /// </summary>
    Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId);
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
TechnologicalOperationService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalOperationService<T, TLink, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceTreeService<T, TContext>(contextFactory), ITechnologicalOperationService<T, TLink>
    where T : ReferenceTreeBase<T>, ITechnologicalOperation, IDomainObjectHasKey<Guid>, new()
    where TLink : TechnologicalOperationUnitBase<T>
    where TContext : DbContext
{
    public async Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        // LINQ-Ct0Cö@CäA CäBD 0C 0D$KC$OCTB C,,4CT0C`LCö>:D
        // A ;C 3Cä4C @Dö BCä<D2Ä GD$> C" FV6†æöÆöv-6 Ä÷ W& F-öäVæ-D& 6R AC$>C"AD$2Cä Væ-B
        // C,,<CT5D" AD$@Cä3C,,9 D$8Cö Væ-D& 6RÄ Tb 6÷&R ?CäAD$@Cä8D" GC,,AD$KC' 5 Äö=ö"ä 0C$BCä<C BC,,GCTAC=8.D
    }
}

```

```

        return await context.Set<TLink>()
            .Where(x => x.OperationId == operationId)
            .Select(x => x.Unit)
            .AsNoTracking()
            .ToListAsync();
    }
}

/// <summary>
/// AÄAD$0C$;Dö5CÄ <CTBCä4 C 1D BD 0C=BCÖKCÄ =C BCT:D4ICT< D4@Cä2CÖ5. A= CÖ:D 5D$=D´9 Cö@C,,:C´0CD=Cä9 D
/// D$5DT?D >Dd5D ACä2 D 0CÄ ?CT@CT>Cö@CT4CT;C,,B CT3Cä 4C´O D >Ct4C =C,,O D 2Cä8DR ?Cä;C,,<Cä@DD=D´E Cä?CT@
/// </summary>
public abstract override Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// AÖ0C"BC, :Cä=C@CTBCÖCDä 8CÖAD$@D4:Dd8Dä @C ADt5D$0 CD;Dö BD >C":Cf C >D CCD>C$0CÖ8CR ² Cö5D 0Dd8D
    /// </summary>
    Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId);
}

/// <summary>
/// Aö>C´CDt8D$L C$ACR <CTBCä4D² @C ADt5D$0 CD;Dö :Cä=C@CTBCÖ>C´ >Cö5D 0Dd8C, =C >C >D CCD>C$0CÖ8C•
/// </summary>
Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter,
TContext>()
    IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalParameterCalcMethodService<T,
TUnit, TOperation, TParameter>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>()
            .FirstOrDefaultAsync(x => x.UnitId == unitId &&
                x.TechnologicalOperationId == operationId &&
                x.TechnologicalParameterId == parameterId);
    }
}

public async Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId)
{

```

```

        await using TContext context = await ContextFactory.CreateDbContextAsync();
    }
    return await context.Set<T>()
        .Where(x => x.TechnologicalOperationId == operationId && x.UnitId == unitId)
        .Include(x => x.TechnologicalParameter) // A0>CD3D CCd0CT< D 2D07C =C0KC' ?C @C <CTBD
        .AsNoTracking()
        .ToListAsync();
    }
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
ITechnologicalParameterService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterService<T> : IReferenceService<T>
    where T : TechnologicalParameterBase
{
    /// <summary>
    /// A0>C`CDt8D$L C00D 0CA5D$@D²Â >D$DC,,;DÄBD >C$0C0=D`5 C0> D$8C0C CD0C0=D`E (Numeric, String, Boolean, D
    /// </summary>
    Task<List<T>> GetByDataTypeAsync(string dataType);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
TechnologicalParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterService<T, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceService<T, TContext>(contextFactory), ITechnologicalParameterService<T>
    where T : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByDataTypeAsync(string dataType)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.DataType == dataType)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
/// <summary>
/// A0>C0BD 0C0B D 5D 2C,,AC0>C4> D ;Cä0 CD;D0 CC0@C 2C`5C08D0 AC0@C 2CäGC08C0>CÄ 5CD8C08Db 8Ct<CT@CT=C,,0.0
/// </summary>
public interface IUnitOfMeasurementService : IReferenceService<UnitOfMeasurement>
{
}
</file>

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;

```

```

/// <summary>Ð
/// B 5D 2C,,A D4?D 0C$;CT=C,,O CÔ>D <C BC,,2CÔ>-D ?D 0C$>Dt=Cä9 C,,=DD>D <C FC,,5C' 5CD8CÔ8Db 8Ct<CT@CT=C,,O.Ð
/// </summary>Ð
public class UnitOfMeasurementService(IDbContextFactory<MesApplicationDbContext> contextFactory)Ð
    : ReferenceService<UnitOfMeasurement, MesApplicationDbContext>(contextFactory),
    IUnitOfMeasurementServiceÐ
{Ð
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public interface IUnitBaseService<TContext> : IReferenceTreeService<UnitBase, TContext> // <- A,,!Aô A A' Aô
D$8C00 C" @Cä4C,,BCT;DÄAC=8C' 8CÔBCT@DD5C"A!Ð
    where TContext : DbContextÐ
{Ð
    Task<List<UnitBase>> GetByKindAsync(UnitKind kind);Ð
    Task<List<UnitBase>> GetByTypeAsync(UnitType type);Ð
    Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type);Ð
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/UnitBaseService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public abstract class UnitBaseService<TContext>(IDbContextFactory<TContext> contextFactory)Ð
    : ReferenceTreeService<UnitBase, TContext>(contextFactory), IUnitBaseService<TContext>Ð
    where TContext : DbContextÐ
{Ð
    // =====Ð
    // Aô A "BD B AT Aô+A' A !B$ A B$ B' BT#A¢ ,, AT A' At#AT"B / Aô B Aä Aô Ad )Ð
    // =====Ð
    Ð
    /// <summary>Ð
    /// A= >CÔBD 0C=Ð DD0C @C,,:C,â !C <C @CT0C'8Ct0Dd8Dò 1D44CTB CÔ0C08D 0CÔ0 CÔ0 D ;Cä9 CÔ8Cd5 Ð
    /// C" :Cä=C= @CTBCÔ>CÂ ACT@C$8D 5, C= >D$>D KC' 2C,,4C,,B CÔ>C'8CÄ>D DCÔKCR :C'0D AD² ,,FW 'FÖVçEVæ-B 8 CD@.
    /// </summary>Ð
    public abstract override Task<UnitBase> CreateChildTemplateAsync(UnitBase parent);Ð
    Ð
    // =====Ð
    // Bt B "B' AÄ B$ AD+ A$+A B A, A Aô+BR ,,!C$>C"AD$2C ¶-æB 8 Type D$5C05D L C$8CD=D² 8CD5C ;DÄ=Cä•
    // =====Ð
    Ð
    public async Task<List<UnitBase>> GetByKindAsync(UnitKind kind)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        return await context.Set<UnitBase>().Ð
            .AsNoTracking()Ð
            .Where(x => x.Kind == kind)Ð
            .ToListAsync();Ð
    }Ð
    Ð
    public async Task<List<UnitBase>> GetByTypeAsync(UnitType type)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        // A,,ACô>C'LctCCT< Cô>C 8D$>C$>CR $ ", D$0C¢ :C : UnitType - DÔBCä 'fÆ w5Y
        return await context.Set<UnitBase>().Ð
            .AsNoTracking()Ð
            .Where(x => (x.Type & type) != 0)Ð
            .ToListAsync();Ð
    }Ð
    Ð
    public async Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð

```

```

        return await context.Set<UnitBase>()
            .AsNoTracking()
            .Where(x => x.Kind == kind && (x.Type & type) != 0)
            .ToListAsync();
    }
}
</file>

<file path="Promatis.Net.MES.Tests/AssemblyInfo.cs">
// AÄBCä;DäGC 5D" ?C @C ;C´5C´LCÖ>CR 2D´?Cä;CÖ5CÖ8CR BCTAD$>C" 2 DÖBCä9 D 1Cä@Cä5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        <IsPackable>false</IsPackable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="coverlet.collector" Version="10.0.0">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
        </PackageReference>
        <PackageReference Include="FluentAssertions" Version="8.10.0" />
        <PackageReference Include="FluentValidation" Version="12.1.1" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
        <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
        <PackageReference Include="Moq" Version="4.20.72" />
        <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
        </PackageReference>
        <PackageReference Include="xunit.v3" Version="3.2.2" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.Mes.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Using Include="Xunit" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs">
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Configuration;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.MES.Service;
namespace Promatis.Net.MES.Tests;
public class UnitServiceStandaloneTests
{
    // 1. A„ACö>C´LCtCCT< C$5Ct4CR FW7DF$6öçFW‡M
    private readonly IDbContextFactory<TestDbContext> _factory;
    private readonly DbContextOptions<TestDbContext> _options;

    public class TestUnit : Domain.UnitBase { }
}

```

```

/// <summary>ð
/// B$5D BCä2D´9 D 5D 2C,,A CD;Dò 2CT@C,,DC,,:C FC,,8 C 8Ct=CTA-C$KC >D >C¢ Væ-D& 6R 2 CÄ>CDCC´LCÔKDR BCTAD$0
/// A,,!Aô A´ Aô : B 8C4=C BD4@C ?D 8C$5CD5C00 C¢ AD$0C04C @D$C UnitBaseService D >CD=C,,< generic-C00
/// </summary>ð
public class TestUnitService(IDbContextFactory<TestDbContext> factory)ð
    : UnitBaseService<TestDbContext>(factory)ð
{ð
    /// <summary>ð
    /// Aô@CäAD$5C"HC 0 Ct0C4;D4HC=0 DD0C @C,,:C, ?Cä;C,,<Cä@DD8Ct<C 4C´O D4AC05D,,=Cä9 C= >CÄ?C,,;D0FC,,8 C 1
    /// </summary>ð
    public override Task<Domain.UnitBase> CreateChildTemplateAsync(Domain.UnitBase parent)ð
    {ð
        // A,,!Aô A´ Aô : A 5Ct>C00D =Cä5 CD8C00CÄ8Dt5D :Cä5 D >Ct4C =C,,5 C,,=D BC =D 0 C00D$8C$=Cä3Cä B
        var childMock = (Domain.UnitBase)Activator.CreateInstance(parent.GetType());ð

        childMock.ParentId = parent.Id;ð
        childMock.Parent = null;ð

        return Task.FromResult(childMock);ð
    }ð
}ð

public UnitServiceStandaloneTests()ð
{ð
    // 2. Aä?Dd8C, 4Cä;Cd=D² 1D´BDÄ 4C´O TestDbContextð
    _options = new DbContextOptionsBuilder<TestDbContext>()ð
        .UseInMemoryDatabase(Guid.NewGuid().ToString())ð
        .Options;ð

    // 3. AÄ>C=0CT< DD0C @C,,:D2 8CÄ5C0=Cä 4C´O TestDbContextð
    var factoryMock = new Mock<IDbContextFactory<TestDbContext>>();ð

    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))ð
        .ReturnsAsync(() => new TestDbContext(_options)); // ReturnsAsync D4?D >D"0CTB Task.FromResultð

    _factory = factoryMock.Object;ð
}ð

// 4. A= >C0BCT:D B D$>Cd5 CD>C´6CT= C0@C,,=C,,<C BDÄ ?D 0C$8C´LCÔKCR >C0FC,,8ð
public class TestDbContext(DbContextOptions<TestDbContext> options)ð
    : ApplicationDbContext(options, new ConfigurationBuilder().Build())ð
{ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)ð
    {ð
        base.OnModelCreating(modelBuilder);ð

        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();ð
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");ð
    }ð

    public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken =
default)ð
    {ð
        IEnumerable<EntityEntry<Domain.UnitBase>> entries =
ChangeTracker.Entries<Domain.UnitBase>()ð
            .Where(e => e.State is EntityState.Added or EntityState.Modified);ð

        foreach (EntityEntry<Domain.UnitBase> entry in entries)ð
        {ð
            Domain.UnitBase unit = entry.Entity;ð
            if (unit.ParentId.HasValue)ð
            {ð
                // At0C4@D46C 5CÄ @Cä4C,,BCT;Dò 4C´O C0@Cä2CT@C=8 (C" -äÖV0÷'´ f-æB @C 1CäBC 5D" 1D´AD$@Cä
                Domain.UnitBase? parent = Set<Domain.UnitBase>().Find(unit.ParentId.Value);ð
                if (parent != null)ð
                {ð
                    // 1. Aô@C 2C,,;Cä ÷6-F-öâ ,, "CT@CÄ8C00C´LCÔKC´ CCT5C²•
                    if (parent.Kind == UnitKind.Position)ð
                    {
                        throw new InvalidOperationException("Position node cannot have
children.");ð
                    }

                    // 2. Aô@C 2C,,;Cä FW 'FÖVÇB ,, CR <Cä6CTB D >CD5D 6C BDÄ ÷6-F-öâ•
                    if (parent.Kind == UnitKind.Department && unit.Kind == UnitKind.Position)ð
                    {
                        throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cf¢ >C JCT:D" :C B
                    }
                }
            }
        }
    }
}ð

```

```

        // 3. Aô@C 2C,,;Câ 8Ct>C´ODd8C, ... &öGV7F-öâö7F÷& vRöG& ç7 ÷'B =CR ACÄ5D,,8C$0DäBD 0)Ð
        if (parent.Kind is UnitKind.Production or UnitKind.Storage or
UnitKind.Transport)Ð
        {Ð
            if (unit.Kind != parent.Kind && unit.Kind != UnitKind.Position)Ð
                throw new InvalidOperationException($"Aô0D CD,,5C08CR 8CT@C @DT8Cfç w·Væ-Bä¶-æ
'{parent.Kind}'.");Ð
        }Ð
    }Ð
}Ð
}Ð
    }Ð
    return await base.SaveChangesAsync(cancellationToken);Ð
}Ð
Ð
Ð
Ð
[Fact]Ð
public async Task GetByKindAsync_Should_Return_Units_Matching_Complex_Mask()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð
    Ð
    // B0GCT9C00 C$ECä4C,,B C" <C AC0C StorageÐ
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Cell_01", Kind =
UnitKind.Storage, Type = UnitType.Cell });Ð
    // B BC =Cä: C$ECä4C,,B C" <C AC0C ProductionÐ
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Lathe_01", Kind =
UnitKind.Production, Type = UnitType.MachineTool });Ð
    Ð
    // ActÐ
    List<Domain.UnitBase> storageUnits = await service.GetByKindAsync(UnitKind.Storage);Ð
    Ð
    // AssertÐ
    storageUnits.Should().ContainSingle(x => x.Name == "Cell_01");Ð
    storageUnits.Should().NotContain(x => x.Name == "Lathe_01");Ð
}Ð
Ð
[Fact]Ð
public async Task GetByTypeAsync_Should_Handle_Specific_Flags()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Crane_1", Kind =
UnitKind.Storage, Type = UnitType.Crane });Ð
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Vehicle_1", Kind =
UnitKind.Transport, Type = UnitType.Vehicle });Ð
    Ð
    // ActÐ
    List<Domain.UnitBase> result = await service.GetByTypeAsync(UnitType.Crane);Ð
    Ð
    // AssertÐ
    result.Should().ContainSingle(x => x.Name == "Crane_1");Ð
}Ð
Ð
[Fact]Ð
public async Task MoveAsync_Should_Throw_When_Moving_Into_Position()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð
    var posId = Guid.NewGuid();Ð
    var machineId = Guid.NewGuid();Ð
    Ð
    await service.AddAsync(new TestUnit { Id = posId, Name = "Terminal Point", Kind =
UnitKind.Position, Type = UnitType.Other });Ð
    await service.AddAsync(new TestUnit { Id = machineId, Name = "Mobile Machine", Kind =
UnitKind.Production, Type = UnitType.MachineTool });Ð
    Ð
    // Act & AssertÐ
    Func<Task> act = async () => await service.MoveAsync(machineId, posId);Ð
    Ð
    await act.Should().ThrowAsync<InvalidOperationException>()Ð
        .WithMessage("Position node cannot have children.");Ð
}Ð
Ð

```

```

[Fact]
public async Task MoveAsync_Should_Prevent_Circular_Dependencies()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();

    // AD>C 0C$;D05CÂ ¶-æB 8 Type, D$0C¢ :C : Cä=C, >C 0Ct0D$5C`LCÔK (required / init)
    await service.AddAsync(new TestUnit
    {
        Id = idA,
        Name = "Node A",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = idB,
        Name = "Node B",
        ParentId = idA,
        Kind = UnitKind.Production,
        Type = UnitType.Section
    });

    // Act & Assert
    Func<Task> act = async () => await service.MoveAsync(idA, idB);

    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("*Bd8C¢;C,,GCTAC¢0D0 7C 2C,,AC,,<CäAD$L*");
}

[Fact]
public async Task GetFullTreeAsync_Should_Maintain_Hierarchy_And_References()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var rootId = Guid.NewGuid();
    var sectionId = Guid.NewGuid();

    await service.AddAsync(new TestUnit
    {
        Id = rootId,
        Name = "Dept",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = sectionId,
        Name = "Section",
        Kind = UnitKind.Production,
        Type = UnitType.Section,
        ParentId = rootId
    });

    await service.AddAsync(new TestUnit
    {
        Id = Guid.NewGuid(),
        Name = "Workstation",
        Kind = UnitKind.Production,
        Type = UnitType.Workstation,
        ParentId = sectionId
    });

    // Act
    Domain.UnitBase? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    tree.Should().NotBeNull();
    tree!.Children.Should().HaveCount(1);
    tree.Children.First().Children.Should().HaveCount(1);
}

```

```

[Fact]
public async Task MoveAsync_Department_Should_Not_Contain_Position()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var deptId = Guid.NewGuid();
    var positionId = Guid.NewGuid();

    // 1. B >Ct4C 5CÂ CT?C @D$0CÄ5CÔB
    await service.AddAsync(new TestUnit
    {
        Id = deptId,
        Name = "Main Dept",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    // 2. B >Ct4C 5CÂ Cä7C,,FC,,N (D >C4;C ACÔ> D$2Cä5C' <C AC5, DÔBCâ <Cä6CTB C KD$L Cell)
    await service.AddAsync(new TestUnit
    {
        Id = positionId,
        Name = "Work Point 1",
        Kind = UnitKind.Position,
        Type = UnitType.Cell
    });

    // Act
    // AôKD$0CT<D 0 Cô5D 5CÄ5D BC,,BDÂ ÷6-F-öâ 2 Department
    Func<Task> act = async () => await service.MoveAsync(positionId, deptId);

    // Assert
    // Aä6C,,4C 5CÂ >D,,8C :D2 >D" Væ-D& 6T†-W& &6‡•G&-vvW-
    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage($"*C0D$5C4>D 8C, u ÷6-F-öâr =CR <Cä6CTB C KD$L C$;Cä6CT= C" tFW 'FÖVçBrç""½
    }

[Fact]
public async Task MoveAsync_Production_Should_Contain_Production_Subnode()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var parentProdId = Guid.NewGuid();
    var childProdId = Guid.NewGuid();

    await service.AddAsync(new TestUnit
    {
        Id = parentProdId,
        Name = "Production Area",
        Kind = UnitKind.Production,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = childProdId,
        Name = "Machine Node",
        Kind = UnitKind.Production,
        Type = UnitType.MachineTool
    });

    // Act
    Func<Task> act = async () => await service.MoveAsync(childProdId, parentProdId);

    // Assert
    // At4CTADÂ 8D :C´NDt5CÔ8Dò 1D´BDÂ =CR 4Cä;Cd=CâÂ BC : C0Cç &öGV7F-öâ <Cä6CTB D >CD5D 6C BDÂ &öGV7
    await act.Should().NotThrowAsync();

    Domain.UnitBase? updated = await service.GetByIdAsync(childProdId);
    updated!.ParentId.Should().Be(parentProdId);
}
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs">
using Microsoft.EntityFrameworkCore;

```

```

namespace Promatis.Net.MES.Tests.UnitBase;
{
    // B$5D BCä2D´9 C¤>CÔBCT:D B C$=D4BD 8 Cô@CäAD$@C =D BC$0 C„<CT= D$5D BCä2D
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        // A„ACô>C´LctCCT< C 0Ct>C$KC´ BC„?, DtBCä1D² BD 8C43CT@ CÄ>C2 4CT;C BDÂ 6WCÄVæ-D& 6Sâ,•
        public DbSet<Domain.UnitBase> Units => Set<Domain.UnitBase>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // 1. A00D BD 0C„2C 5CÄ E B „:C : C" >D =Cä2CÔ>CÄ ?D >CT:D$5)
            modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();

            // 2. B4:C 7D´2C 5CÄÂ GD$> TestUnit – DÔBCä =C AC´5CD=C„:D
            modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
        }
    }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnit.cs">
namespace Promatis.Net.MES.Tests.UnitBase;
{
    public class TestUnit : MES.Domain.UnitBase { }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs">
using Promatis.Net.MES.Domain;
{
    namespace Promatis.Net.MES.Tests.UnitBase;
    {
        public class TestUnitValidator : UnitBaseValidator<TestUnit> { }
    }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs">
using FluentAssertions;
using FluentValidation.TestHelper;
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain.Interface;
{
    namespace Promatis.Net.MES.Tests.UnitBase;
    {
        public class UnitBaseArchitectureTests
        {
            private readonly TestUnitValidator _validator = new();
            private readonly DbContextOptions<AppDbContext> _options;

            public UnitBaseArchitectureTests()
            {
                _options = new DbContextOptionsBuilder<AppDbContext>().
                    .UseInMemoryDatabase(Guid.NewGuid().ToString())
                    .Options;
            }

            private static EntityCancelEventArgs<MES.Domain.UnitBase> CreateArgs(MES.Domain.UnitBase
entity, DbContext context)
            {
                return new EntityCancelEventArgs<MES.Domain.UnitBase>()
                {
                    entity,
                    EntityStateChangeEnum.Added,
                    new List<PropertyChangeInfo>(),
                    context
                };
            }

            #region A$0C´8CD0Dd8Dò „ C AC¤8 C, 1C 7Cä2D´5 Cô@C 2C„;C •

            [Theory]
            [InlineData(UnitKind.Storage, UnitType.Cell, true)] // BôGCT9C¤0 C" !C¤;C 4CR r C-
            [InlineData(UnitKind.Storage, UnitType.Workshop, false)] // Bd5DR 2 B :C´0CD5 – AäHC„1C¤0 (C 8D" =CR ACä2
            [InlineData(UnitKind.Production, UnitType.Table, true)] // B BCä; C" D >C„7C$>CDAD$2CR r C-
            [InlineData(UnitKind.Position, UnitType.Other, true)] // Aô@CäGCT5 C" Cä7C„FC„8 – Aä:D
            [InlineData(UnitKind.Position, UnitType.Table, false)] // B BCä; C" Cä7C„FC„8 – AäHC„1C¤0 (D >C4;C ACô>
            public void Validator_Should_Check_Bitwise_Compatibility(UnitKind kind, UnitType type, bool
isValid)

```

```

{
    var unit = new TestUnit { Kind = kind, Type = type, Name = "Test" };
    TestValidationResult<TestUnit>? result = _validator.TestValidate(unit);

    if (isValid)
    {
        result.ShouldNotHaveAnyValidationErrors();
    }
    else
    {
        result.ShouldHaveValidationErrorFor(x => x)
            .WithErrorMessage("B$8Cò =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½");
    }
}

#endregion

#region B$@C,,3C45D ,, D 0C$8C'0 C$;Cä6CT=CÔ>D BC, ¶-æB•

[Fact]
public async Task Trigger_Should_Allow_Root_Units()
{
    await using var context = new AppDbContext(_options);
    var root = new TestUnit { ParentId = null, Kind = UnitKind.Department, Type =
UnitType.Other };

    var trigger = new UnitBaseHierarchyTrigger();
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(root, context);

    await trigger.HandleAsync(args);

    args.Cancel.Should().BeFalse();
}

[Fact]
public async Task Trigger_Should_Reject_Position_In_Department()
{
    await using var context = new AppDbContext(_options);
    var parentId = Guid.NewGuid();

    context.Units.Add(new TestUnit { Id = parentId, Kind = UnitKind.Department, Type =
UnitType.Other, Name = "Dept" });
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestUnit { ParentId = parentId, Kind = UnitKind.Position, Type =
UnitType.Other, Name = "Pos" };
    var trigger = new UnitBaseHierarchyTrigger();
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);

    await trigger.HandleAsync(args);

    args.Cancel.Should().BeTrue();
    args.ErrorMessage.Should().Contain("CÔ5 CÄ>Cd5D" 1D´BDÂ 2C´>Cd5Cò 2 'Dept' (Department));
}

[Theory]
[InlineData(UnitKind.Production, UnitKind.Production, false)] // Bd5DR 2 Bd5DR r C-
[InlineData(UnitKind.Production, UnitKind.Transport, true)] // B$@C =D ?Cä@D" 2 Bd5DR r C ?D 5D"5CÔ> (
[InlineData(UnitKind.Storage, UnitKind.Position, false)] // Aô>Ct8Dd8Dò 2 B :C'0CB r C-
[InlineData(UnitKind.Position, UnitKind.Position, true)] // Aô>Ct8Dd8Dò 2 Aô>Ct8Dd8Dâ r C ?D 5D"5CÔ
public async Task Trigger_Should_Enforce_Isolation(UnitKind pKind, UnitKind cKind, bool
shouldCancel)
{
    await using var context = new AppDbContext(_options);
    var parentId = Guid.NewGuid();

    // B >Ct4C 5CÂ @Cä4C,,BCT;Dò A Cò>CDECä4DôIC,,< D$8Cò>Cí
    context.Units.Add(new TestUnit { Id = parentId, Kind = pKind, Type = GetFirstType(pKind),
Name = "P" });
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestUnit { ParentId = parentId, Kind = cKind, Type = GetFirstType(cKind),
Name = "C" };
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);
    await new UnitBaseHierarchyTrigger().HandleAsync(args);
}

```

```

        args.Cancel.Should().Be(shouldCancel, $"Cö>D$>CÄC DtBCâ 2C´>Cd5C08CR ¶4¶-æG0 2 {pKind} CD>C´6C0> C0@C
    }
}
#endregion
}
private static UnitType GetFirstType(UnitKind kind)
    => Enum.GetValues<UnitType>().First(t => t != UnitType.None && ((int)kind & (int)t) != 0);
}
</file>

<file path="Promatis.Net.MES.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspaces
@using Promatis.Net.UI.Components.EditDialogs
</file>

<file path="Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="UnitBase\*" />
        <Content Remove="UnitBase\*" />
        <EmbeddedResource Remove="UnitBase\*" />
        <None Remove="UnitBase\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /
    >
        <ProjectReference Include="..\..\Core\Promatis.Net.UI\Promatis.Net.UI.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Folder Include="Pages\UnitOfMeasurements\Card\" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.UI/UiModule.cs">
using MudBlazor;
using Promatis.Net.UI;
namespace Promatis.Net.MES.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // A0;CäAC²8C' AC0@C 2CäGC08C¢ BCTEC0>C´>C48Dt5D :C„E C00D 0CÄ5D$@Cä2D
            ("AT4C„=C„FD² 8Ct<CT@CT=C„0", "/mes/mdm/units-of-measurement", Icons.Material.Filled.SettingsInput
        };
    }
}

```

```
    }  
  }  
</file>  
  
<file path="Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js">  
  // This is a JavaScript module that is loaded on demand. It can export any number of  
  // functions, and may import other JavaScript modules if required.  
    
  export function showPrompt(message) {  
    return prompt(message, 'Type anything here');  
  }  
</file>  
  
</files>
```